

Switcher

Projekt Entwurf Eingebetteter Systeme + Master Project Software Engineering of Embedded Systems

Abdelrahman Khaled Mansour M. Elkholy

Andreas Flohr

Johannes Hammermann

Waleed Khaled Fahmy Mohammed

Nils Rheinländer

Adrian Tirpak

Technische Universität Berlin

Abstract

This project presents a LEGO-based robotic manipulator that learns to locate and flip a lever using the vision-language-action model Octo. A custom demonstration dataset was recorded, converted into the RLDS format, and used for model fine-tuning. The project covers motor control, data collection, human–robot interaction, and vision-language-action learning, resulting in a robot capable of executing the target manipulation task from visual observations and language instructions.

1. Server Architecture & Web UI

Multi-layer server architecture

The Python server uses FastAPI to handle API connections and serve the Web UI. A central "Command Executor" routes incoming and outgoing robot commands/tasks to the designated worker threads.

Hardware abstraction layer

To enable cross-platform development, the server implements two interchangeable HALs, selected based on whether a Raspberry Pi and BuildHAT are detected.

- **BuildHAT HAL** for deployment on the physical robot.
- **Simulation HAL** for local testing and simulation.

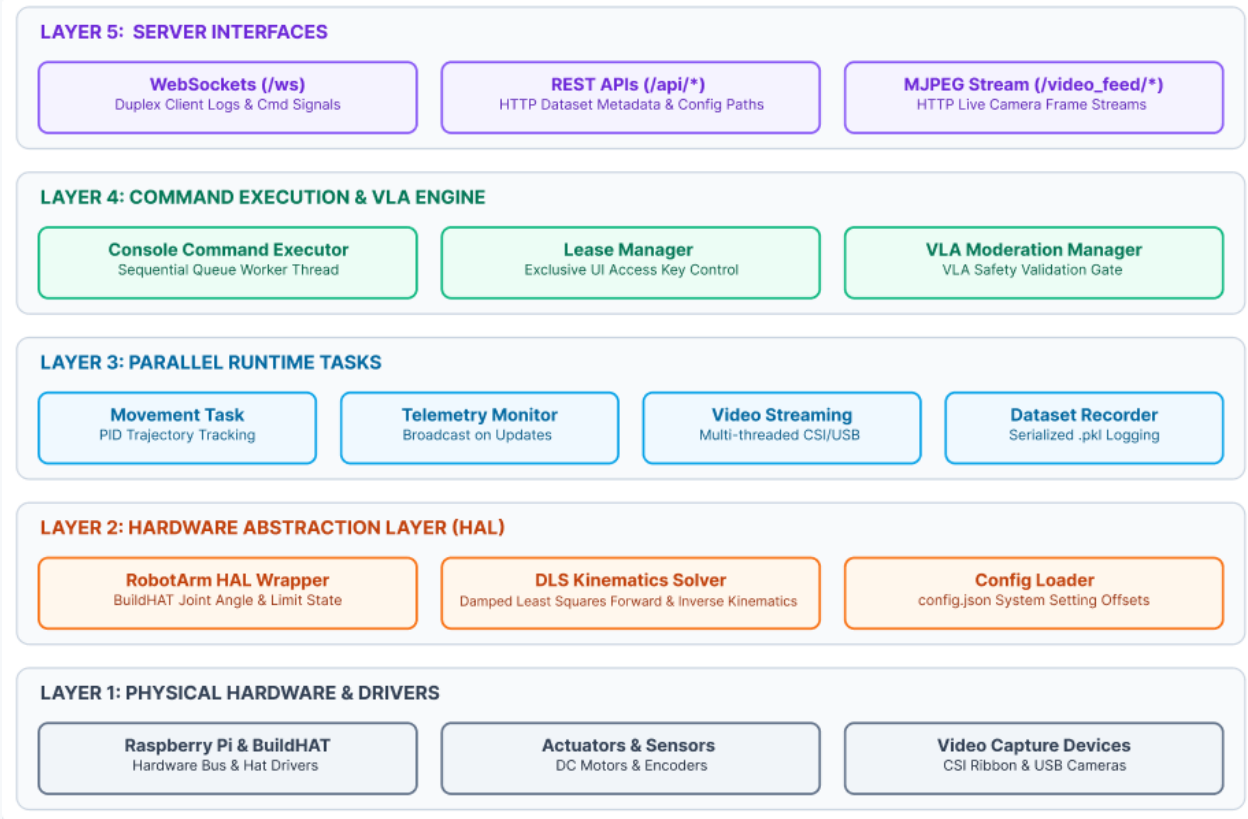


Fig. 2. Multi-layer FastAPI, command-executor, and HAL architecture.

Web interface

- **Frontend:** The user interface is build with Svelte and native CSS, and is used as the main control panel.
- **I/O teleoperation:** To control the robot users can request a exclusive control leases for keyboard, Xbox controller.
- **Data manager:** To browse, inspect, and analyze recorded episodes.
- **Real-time communication:** Telemetry, commands, and state updates are transmitted bi-directionally over WebSockets.

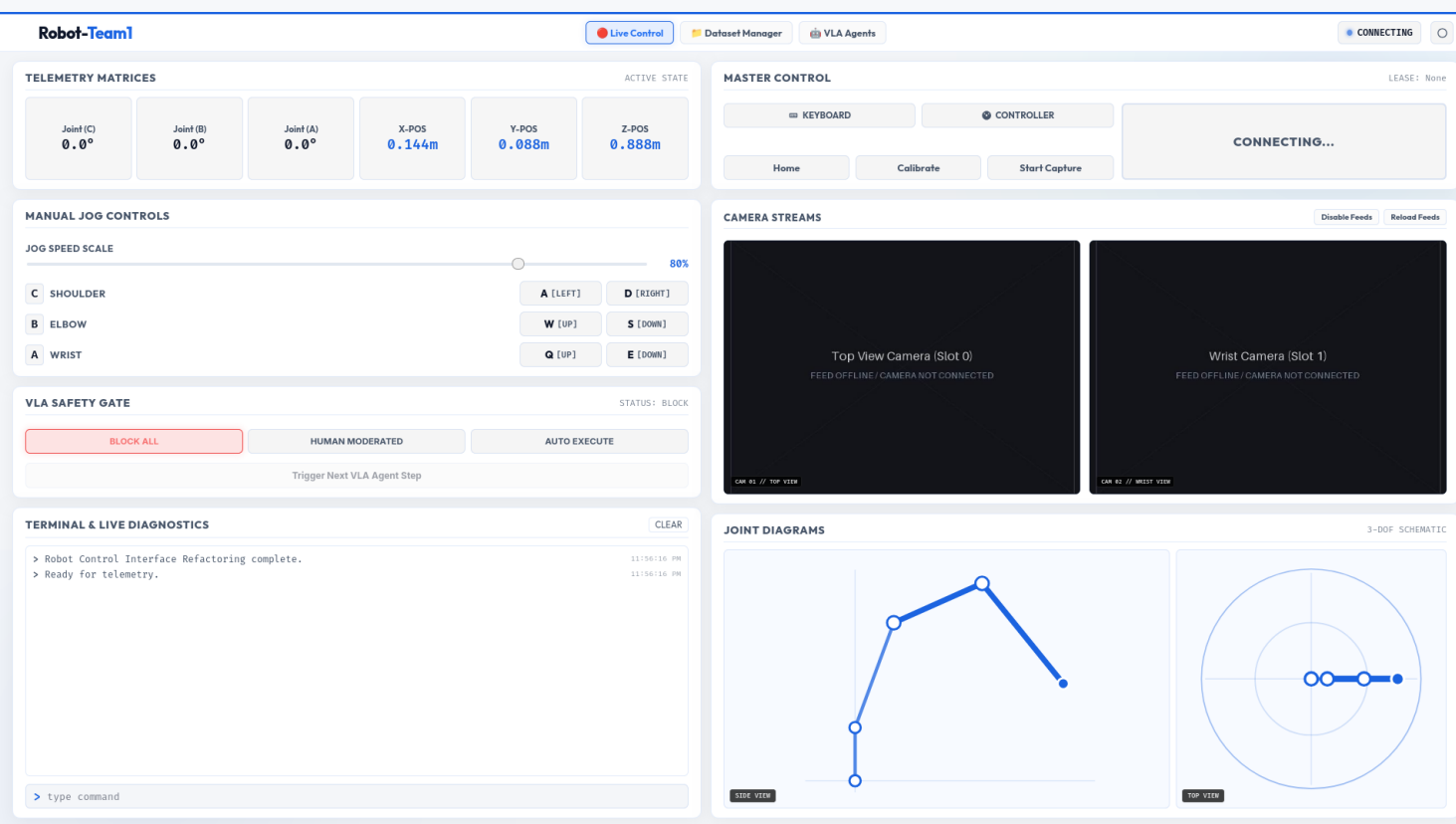


Fig. 3. Dashboard with telemetry, cameras, manual controls, emergency stop, joint diagrams, and episode replay.

2. Teleoperation, Replay & Test Plan

An Xbox controller is used for teleoperation. The client maps one joystick axis to each motor, enabling simultaneous, nuanced control. Every input passes through normalization, deadzone filtering, and input smoothing. The processed values are converted into motor jog commands and transmitted to the Raspberry Pi server over WebSocket. To ensure operational safety, an emergency stop function is mapped to the trigger buttons, allowing the operator to halt robot motion immediately. Additional controller buttons are assigned to joint calibration and trajectory recording functions.

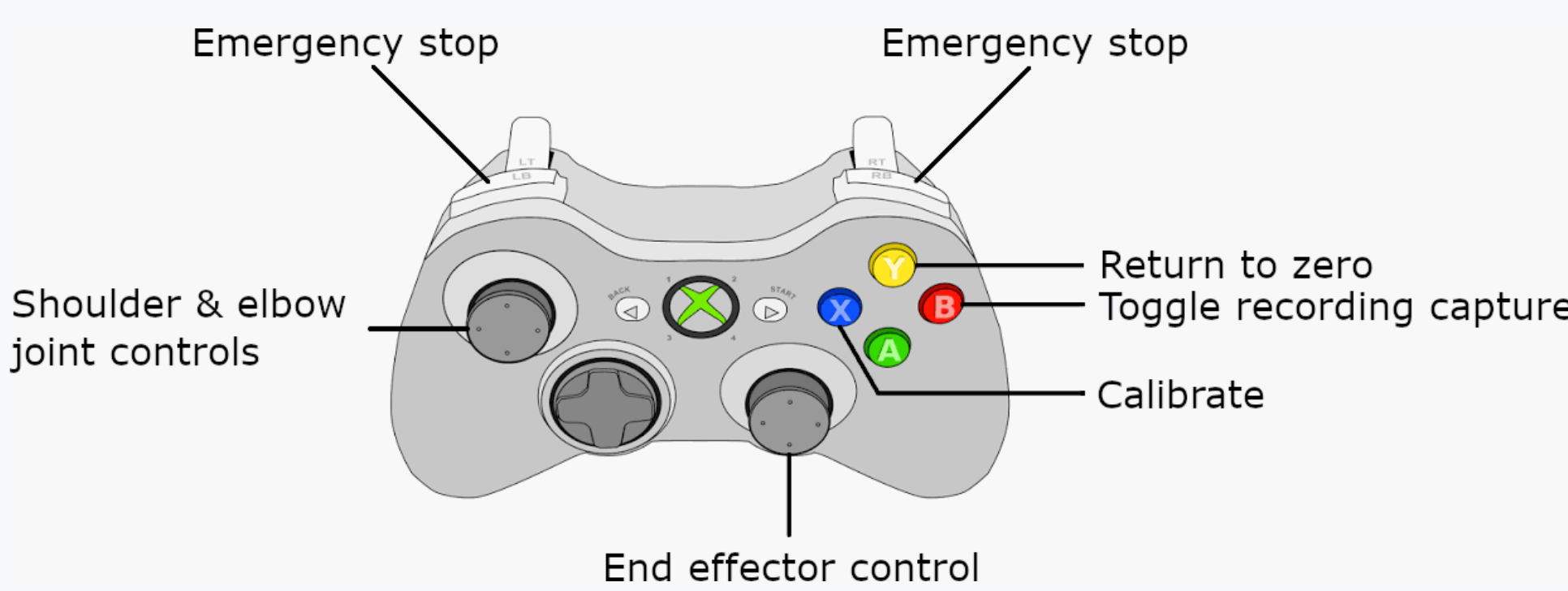


Fig. 4. Teleoperation control scheme with joint controls & additional features.

The user interface provides replay functionality for executing previously recorded demonstrations on the robot arm. The replay system first extracts the recorded starting position and moves the robot to that configuration. It then sequentially replays the recorded trajectory by executing each stored motion command. This allows recorded demonstrations to be validated before they are transformed into a dataset and used for model training.



Fig. 5. Test categories.

The project contains a test plan which is divided into six segments with a focus on hardware stability, robot safety, and motor control validation. A total of 17 tests are executed throughout the development process, ensuring that new features do not introduce regressions.

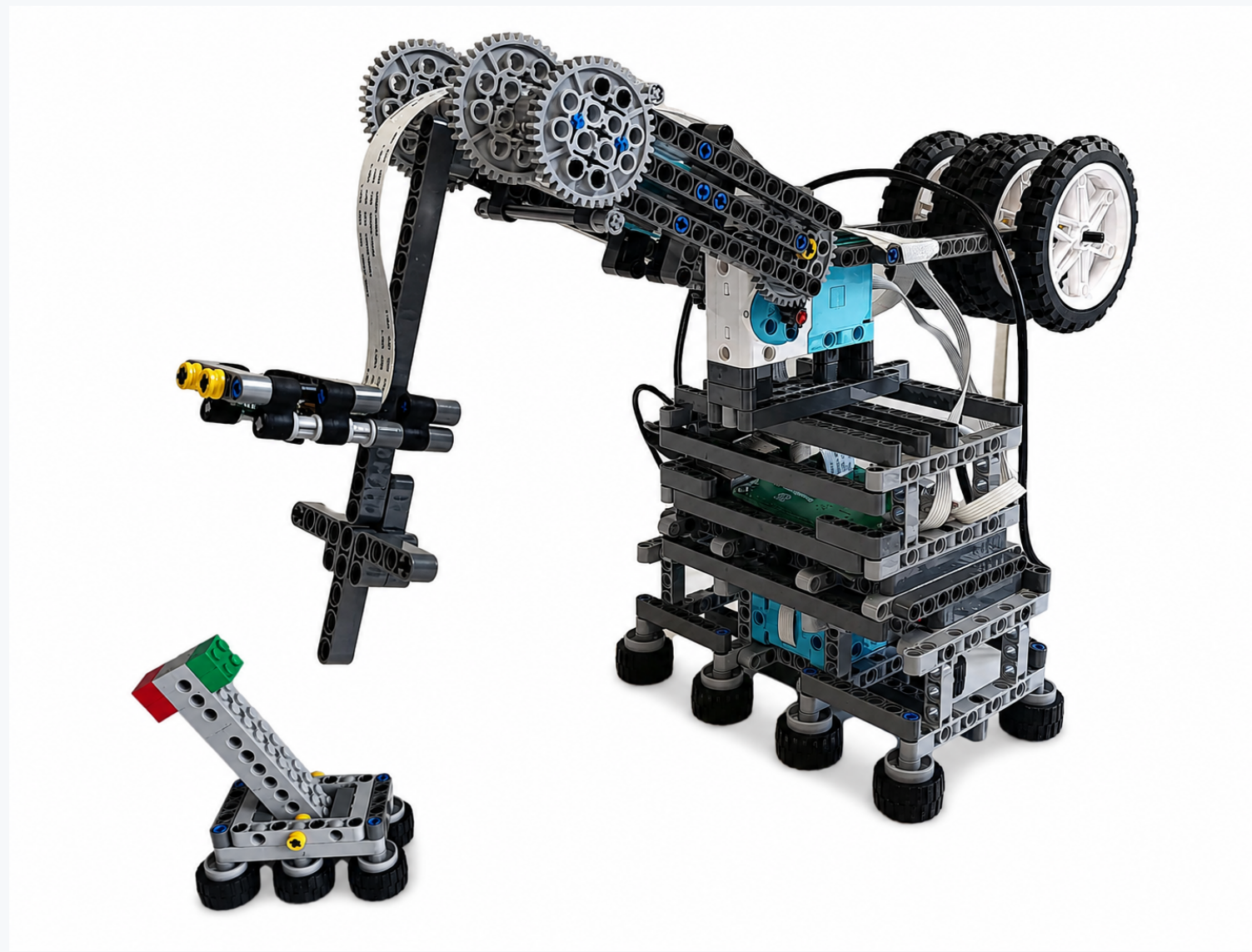


Fig. 1. Annotated view of the LEGO manipulator, cameras, and lever.

3. Robot Architecture & Movement

The manipulator has three degrees of freedom and is built from LEGO components with motors controlled by a BuildHAT. Link geometry is measured and stored in a URDF model. It supports forward kinematics during data collection and inverse kinematics during inference.

Movement strategy

- Controller and keyboard inputs are scaled by direction and converted to a fraction of maximum motor power using PWM.
- Upward motions receive greater torque to compensate for gravity.
- During inference, Octo end-effector coordinates are converted into target joint motion using inverse kinematics.
- New target commands supersede unfinished targets instead of entering a queue, keeping the arm responsive.

4. Motion Control & Kinematics

Direction-aware PID position control

Closed-loop PID position control is implemented for all three motors. Continuous encoder feedback corrects position errors, while direction-specific positive and negative gains compensate for asymmetric mechanical loads. Filtered encoder velocity supplies derivative feedback:

$$u_k = K_{p,\pm} e_k + K_{i,\pm} \sum e_i \Delta t - K_{d,\pm} \dot{q}_k, \quad u_k \in [u_{\min}, u_{\max}],$$

where $e_k = q_{\text{target}} - q_{\text{cont}}$ and u_k is the commanded motor PWM. The controller also applies integral clamping, saturation-aware anti-windup, stiction compensation, a position-and-velocity settling criterion, and progressive slowdown near software joint limits.

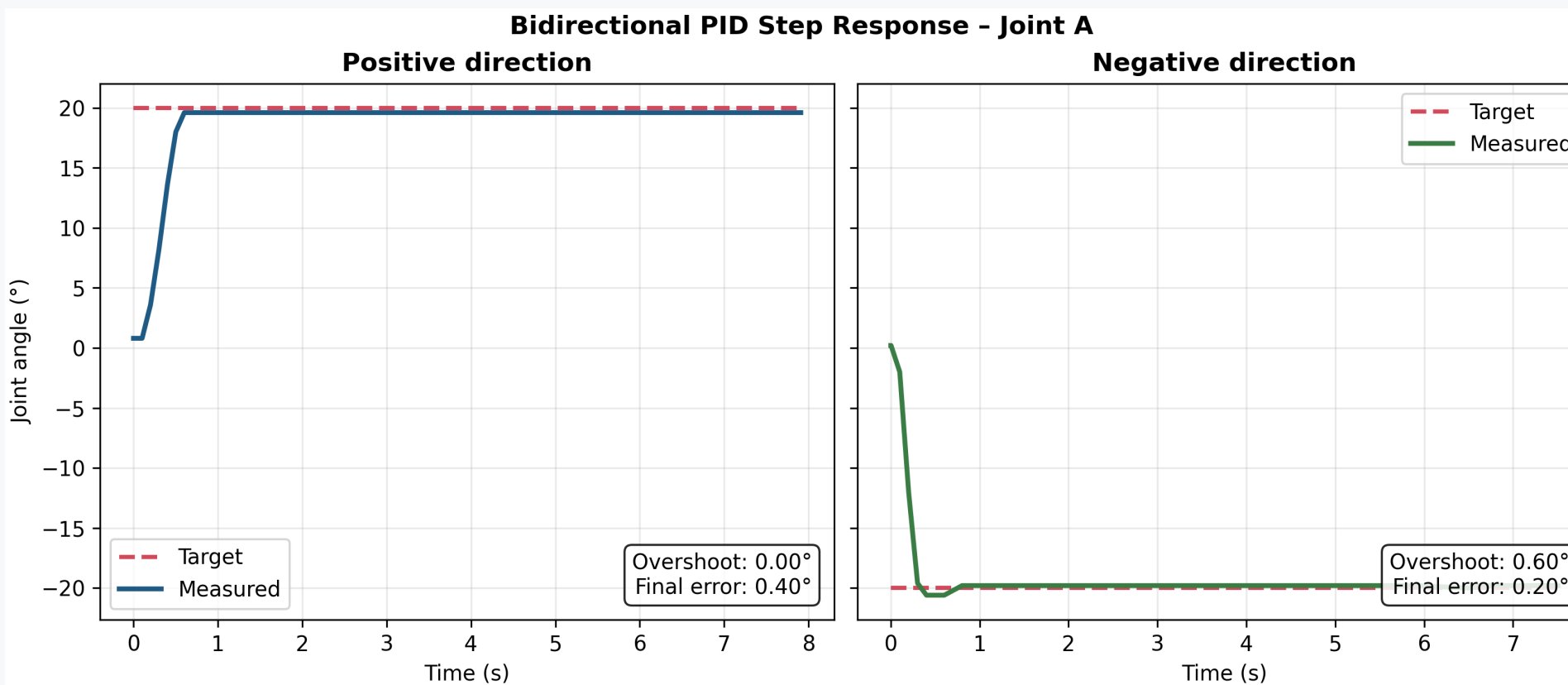


Fig. 6. Bidirectional PID response for joint A with pwm 0.6.

Continuous encoder reconstruction

The absolute motor encoders wrap within $q_{\text{abs}} \in [-180^\circ, 180^\circ]$. A continuous motor position is reconstructed as

$$q_{\text{cont}} = q_{\text{abs}} - q_0 + 360^\circ N,$$

where q_0 is the calibrated home offset and N is the tracked revolution count. The count is updated whenever the difference between consecutive raw encoder samples crosses $\pm 180^\circ$.

Damped-Least-Squares Inverse Kinematics

Pinocchio evaluates the forward kinematics from the URDF model. For Cartesian end-effector targets, an iterative damped least-squares (DLS) inverse kinematics algorithm computes the joint velocity update:

$$\Delta q = J^T (J J^T + \lambda^2 I)^{-1} (x_{\text{goal}} - x_{\text{current}}),$$

where J is the translational Jacobian and λ denotes the damping factor. To ensure smooth convergence of the inverse kinematics solver, the joint configuration is iteratively updated via

$$q_{n+1} = q_n + \alpha \Delta q,$$

for a maximum of $n_{\text{max}} = 150$ steps, where $\alpha = 0.2$ represents the step gain factor. To ensure the calculated updates remain physically and geometrically valid throughout this iterative process, each joint angle is constrained at every step using the element-wise clamping relationship:

$$q_{n+1} = \max(q_{\min}, \min(q_{n+1}, q_{\max}))$$

5. Calibration, Homing & Safety

Zero calibration and homing

- Physical zero poses are defined using mechanical witness marks.
- Witness-mark alignment is matched with URDF zero configuration.
- Absolute encoder values are stored as persistent home references.
- The first home command in a session uses the stored absolute encoder references.
- Later homing operations use relative differences from the cached session-home motor positions.
- Homing and calibration are accessible through the Web UI.

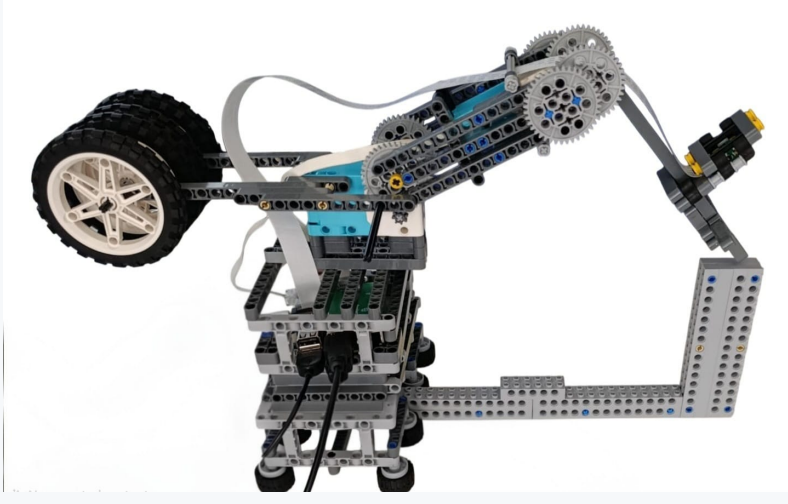


Fig. 7a. Witness mark Motor A (End Effector).



Fig. 7b. Witness mark motor B.

Joint limits and safety

Joint limits are derived from the physical workspace and converted into motor-space through the gear ratios. Target motions and IK solutions are checked against these limits. Manual motion slows progressively and stops near each boundary.

From instruction to action

Observe: primary and wrist RGB cameras, motor encoders.

Interpret: language goal and visual observations enter Octo.

Plan: Cartesian actions pass through constrained inverse kinematics.

Act: direction-aware closed-loop control executes the command.

3 DoF LEGO + BuildHAT	2 RGB primary + wrist
10 Hz dataset capture	RLDS Octo-ready data

6. Data Collection, Preparation & Analysis

Recording

The recorder captures complete task episodes. Each episode contains:

- a goal image,
- a metadata.json file containing the language instruction and episode metadata,
- images from the primary USB camera and the wrist-mounted camera, and
- the calibrated joint angles and the corresponding end-effector positions, stored in per-step pickle files.

During recording, the end-effector positions are computed in real time using forward kinematics based on the calibrated joint angles.

Dataset generation

We recorded a total of 229 demonstration episodes. The data were converted into an RLDS[2] (Reinforcement Learning Datasets) dataset using a custom builder based on the Open X-Embodiment[4] template.

The dataset builder performs the following processing steps:

- converts absolute end-effector poses into delta positions,
- adapts the action and observation representations to our 3-DoF robotic arm,
- transforms the episodes into the format required for Octo fine-tuning, and
- encodes the natural-language task instructions as language embeddings.

Ultimately, the dataset is divided into training and validation sets using a 90/10 split.

Workspace analysis

Motion trajectories are analyzed after collection. In addition, randomly sampled images and recorded motor positions provide a qualitative and quantitative data check.

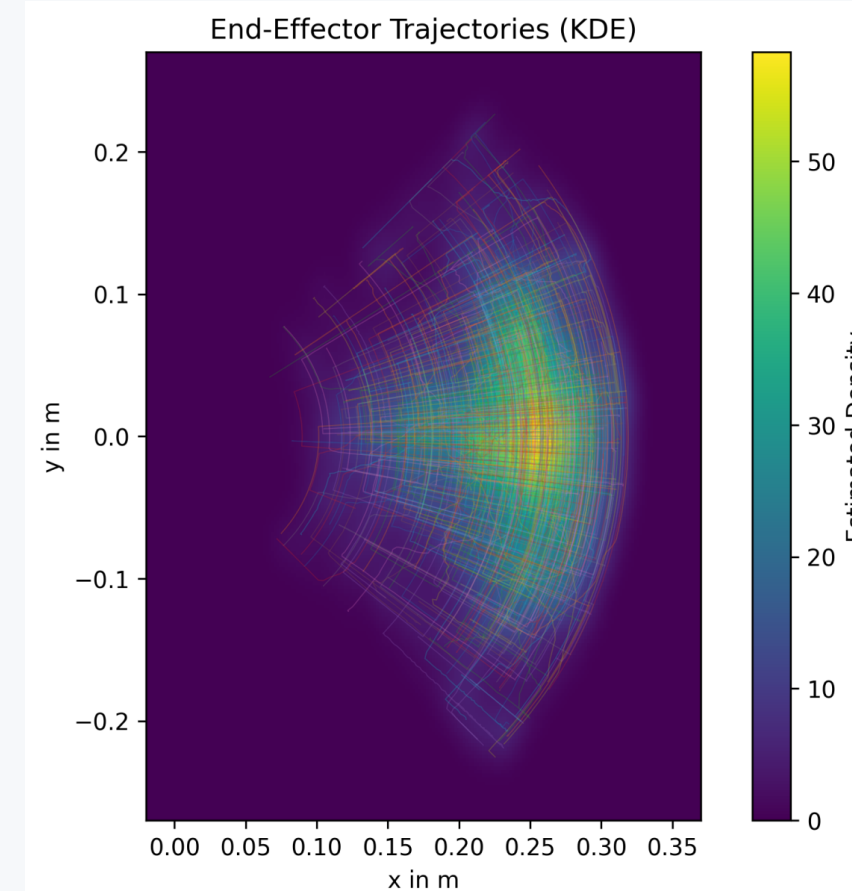


Fig. 8. End-effector trajectory density and sampled paths.

7. Live Inference

The inference client connects to the Web UI over WebSocket, retrieves live camera frames and telemetry, and runs the model. Receding Horizon Control predicts a sequence of steps but executes only a short horizon before re-inferring on a fresh camera frame, ensuring the arm always acts on fresh observations.

8. Octo Fine-Tuning

The Octo pipeline was adapted to the robot's action horizon, with a DiffusionActionHead integrated in place of the default L1 head.

- A one-command Docker wrapper launches reproducible training runs.
- Weights & Biases tracks training loss metrics.



Fig. 9a. Training loss.

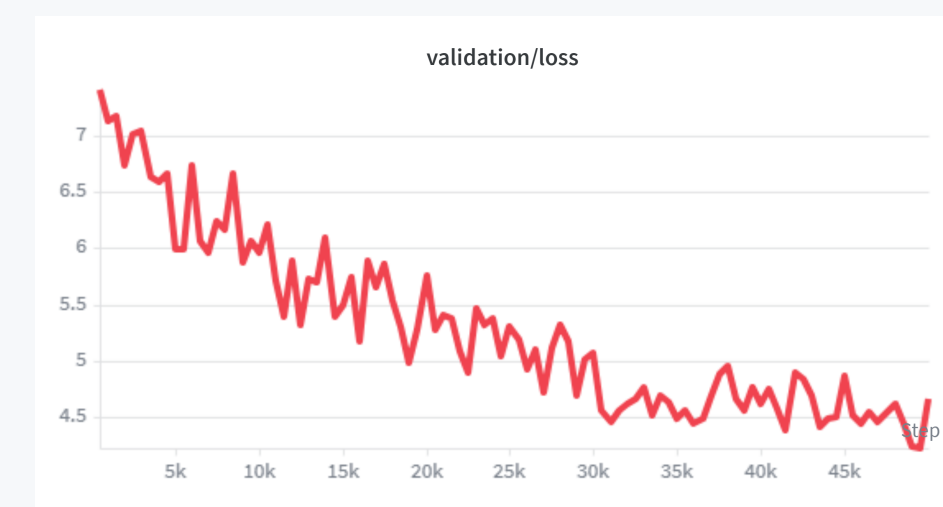


Fig. 9b. Validation loss.

Both training and validation loss decrease over training, indicating the model learns the task and generalizes to held-out episodes rather than overfitting.

9. Results

Offline evaluation compares predicted and ground-truth actions on held-out validation episodes.

Offline result: Fine-tuning reduces the mean action error by roughly 14× compared with the base model, from MAE ≈ 0.131 to ≈ 0.010 .

Live result: The fine-tuned model successfully drives the arm toward the lever during live inference.

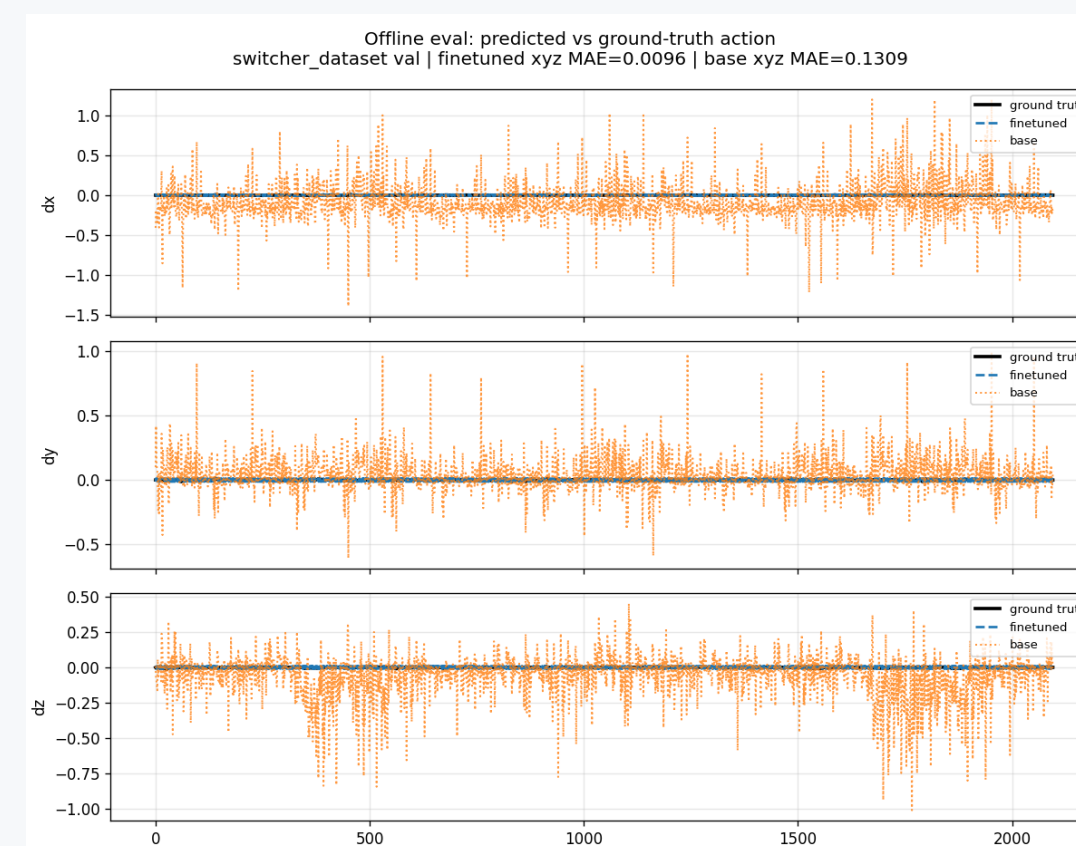


Fig. 10. Predicted vs. ground-truth end-effector motion (dx, dy, dz) on a held-out validation episode.

Team 1: Abdelrahman Khaled Mansour M. Elkholy, Andreas Flohr, Johannes Hammermann, Waleed Khaled Fahmy Mohammed, Nils Rheinländer, and Adrian Tirpak.

Sources: [1] Octo Model Team, *Octo: An Open-Source Generalist Robot Policy*, 2024. [2] S. Ramos et al., *RLDS*, 2021. [3] J. Carpentier et al., *Pinocchio*, 2019. [4] Embodiment Collaboration et al., *Open X-Embodiment: Robotic Learning Datasets and RT-X Models*, 2025. [5] Xbox Controller: https://commons.wikimedia.org/wiki/File:Xbox_Controller.svg